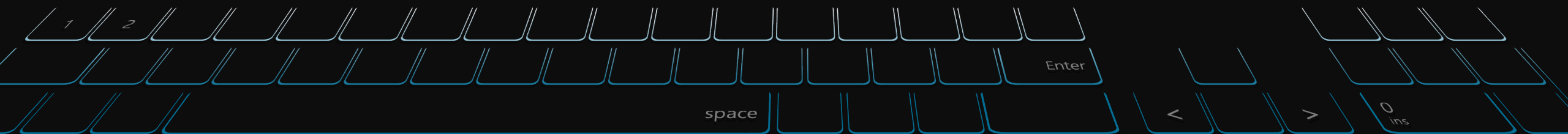


1st Week

Orientation 및 Python 기본, Ipython, Jupyter Notebook



프로필 소개



| AI 현업 및 강의 진행

차성재

무신사 Core AI PM × 아주대 AI 대학원 겸임교수

Artificial Intelligence (AI) · Product Manager (PM)

금융 · AutoML & 은행/카드/보험 AX

의료 · 컴퓨터 비전(실시간 대장내시경 용종 탐지/진단) & DataOps

교육 · LLM(청담어학원/에이프릴어학원 영어 과제 평가) & AICC

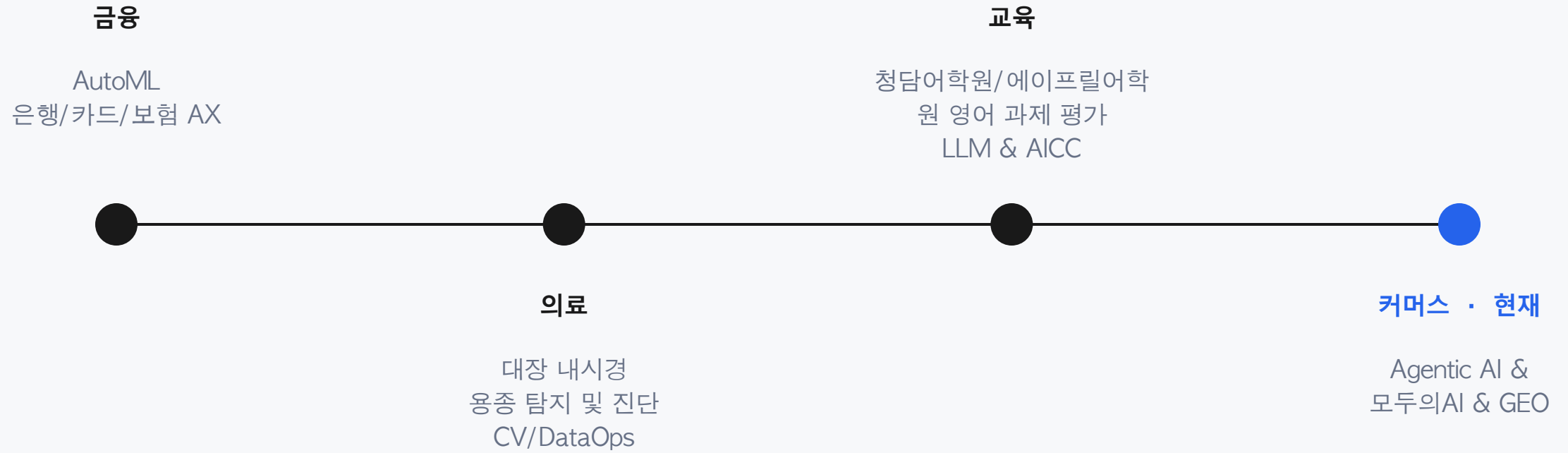
커머스 · Agentic AI & 모두의AI & GEO



문제 정의 · 제품 요구사항 · 평가 · 운영 지표의 연결

PRODUCT · ENGINEERING · EDUCATION

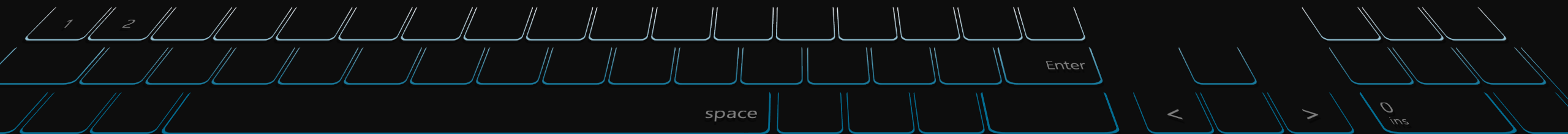
| 정형 데이터에서 생성형 AI까지



산업은 달라도 최신 AI 커리어 패스 도전!

Orientation

- 1) 강의 목표
- 2) 강의 내용
- 3) 강의 타입
- 4) 성적 평가





강의 목표

Objective of Lecture

- 'Python'으로 '데이터 활용 및 분석' 기본기를 갖추는 것!!!
- 그리고 ChatGPT(Codex), Claude와 함께,
우수한 질문을 통해 데이터 분석을 손쉽게 하는 것!

데이터 분석 및 시각화 (GIS 포함)

O'REILLY®

파이썬 라이브러리를 활용한 데이터 분석 3판

Python for Data Analysis

3rd edition



powered by
jupyter

영화 평점, 이름 통계,
선거 데이터 등 실사례 사용

웨스 맥키니 지음
김영근 옮김

한빛미디어
hanbitmedia.com

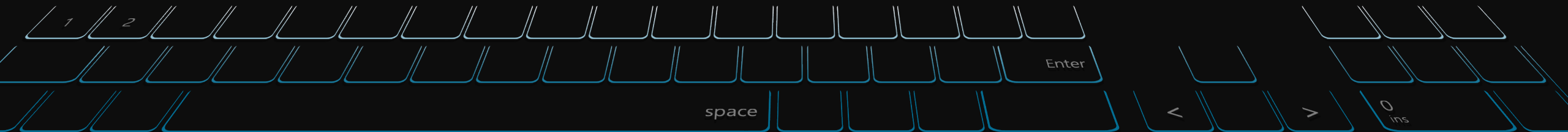


강의 내용

Contents of Lecture

기간	내용	과제
01주차	오리엔테이션 Python 언어의 기본, lpython, 주피터 노트북	-
02주차	내장 자료구조, 함수, 파일	실습 과제
03주차	Numpy 기본 : 배열과 벡터 연산	실습 과제
04주차	Pandas 시작하기	실습 과제
05주차	데이터 로딩과 저장, 파일 형식	실습 과제
06주차	데이터 정제 및 준비(조인, 병합, 변형)	실습 과제
07주차	그래프와 시각화	기말 프로젝트 상세 공지
08주차	중간고사 - 오프라인 오픈북으로 진행	실습 과제

기간	내용	과제
09주차	데이터 집계와 그룹 연산	실습 과제
10주차	시계열	실습 과제
11주차	고급 pandas	실습 과제
12주차	파이썬 모델링 라이브러리	실습 과제
13주차	데이터 분석 예제 / 고급 Numpy	실습 과제
14주차	Kaggle / Dacon 데이터 분석 사례 Case Study	프로젝트 레포트 제출
15주차	[기말고사] Team별 프로젝트 결과 발표	종강~!





강의 타입 (혹은 강의 스타일)

Style of Lecture

- 수업은 교재 위주로 준비
- 수업 시간이 남을 시, 데이터 / 인공지능 분야에 도움이 될 추가 리소스들을 공유하는 방식으로 진행
- **오픈채팅방 '2026-1학기 도시데이터코딩' 으로 참고하면 좋을 자료 공유할 예정 (참여 code = python)**
- AI 산업에 대한 궁금증 혹은 개별 취업/진로 상담이 필요한 사람은 카톡으로 편하게 연락할 것!
 - 대면 / Zoom 모두 가능 (이메일은 답이 느리므로 공식적인 것 포함 최대한 카톡으로 연락주기!)
 - 차성재 강사 연락처
 - 010-9562-9958
 - Kakao ID : sungjai426
 - E-mail : business.sjcha@gmail.com



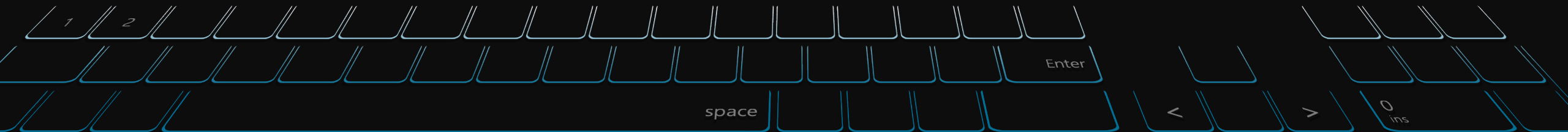
성적 평가

Grade Evaluation

- 출석 10%
- 과제 30% (중간 15% / 기말 15%)
 - 제출 여부 및 결과물 구현을 잘 하였는지 확인하여 불충분시 감점하는 방식으로 평가
- 중간고사 시험 30% (객관식 40개)
 - 채점 기준에 따른 평가
 - 시험은 오프라인 오픈북 (온라인 / 패드 혹은 노트북 기반 PDF 활용 불가!)
- 기말고사 프로젝트 30% (보고서 제출 + 발표) -> 중간고사 이후 프로젝트 내용 상세 공지
 - 프로젝트 보고서 및 발표 수준에 따른 평가 (팀 평가)

1부: *Intro to Study*

- 1) 수업 때 다룰 데이터
- 2) 데이터 분석 with 파이썬
- 3) 필수 파이썬 라이브러리
- 4) PC별 세팅 방법
- 5) 파이썬 Package 설치 및 업데이트





1.1.1 어떤 데이터를 사용하나

여기서 ‘데이터’는 정확히 무슨 뜻일까? 주된 의미는 **구조화된 데이터**다. 일부러 구조화된 데이터라는 모호한 표현을 썼는데, 다음과 같은 여러 가지 형태의 데이터를 포함한다.

- 각 컬럼의 형식이 문자열, 숫자, 날짜 등으로 서로 다른 표 혹은 스프레드시트와 비슷한 데이터. 이는 관계형 데이터베이스 혹은 탭이나 쉼표로 구분되는 텍스트 파일 형식으로 저장되는 대부분의 데이터를 포함한다.
- 다차원 배열(행렬)
- SQL에서 기본키나 외래키 같은 키 컬럼에 의해 서로 연관되는 여러 가지 표
- 일정하거나 일정하지 않은 간격의 시계열



1.2 왜 데이터 분석에 파이썬을 사용하나

파이썬은 매력적인 언어다. 1991년 처음 발표된 이래 펄, 루비 같은 인기 있는 언어가 되었다. 특히 최근 몇 년 사이에 레일즈(루비), 장고(파이썬) 같은 다양한 웹 프레임워크로 웹사이트를 만들면서 파이썬과 루비는 큰 인기를 얻었다. 이런 언어는 스크립트 언어라고 불리는데, 작은 프로그램이나 업무 자동화 스크립트를 빠르고 간단하게 만들 수 있다. 개인적으로 '스크립트 언어'라는 용어를 좋아하지 않는데, 이름 자체에 제대로 된 소프트웨어를 만드는 데는 사용하지 못한다는 의미를 담고 있기 때문이다. 인터프리터 언어 사이에서도 파이썬은 다양한 역사적, 문화적 이유로 인해 방대하고 활동적인 과학 계산 컴퓨팅 커뮤니티에서 사용되고 있다. 지난 10년간 파이썬은 '대안 언어' 위치에서 데이터 과학, 머신러닝 그리고 범용 소프트웨어 개발에 이르기까지 학계와 업계 모두에서 가장 중요한 프로그래밍 언어 중 하나로 성장했다.

파이썬은 데이터 분석과 대화형(인터랙티브) 컴퓨팅, 데이터 시각화에서 자주 사용하는 R, 매트랩^{MATLAB}, SAS, Stata 같은 오픈소스나 상용 언어, 도구와 비교해도 뒤지지 않는다. 최근에는 (pandas나 scikit-learn 같은) 파이썬 라이브러리 지원이 개선되어 데이터 처리 업무에 두각을 나타내고 있다. 파이썬은 범용적인 프로그래밍 언어일 뿐만 아니라 과학 계산용으로도 손색이 없기에 데이터 애플리케이션 개발을 위한 최고의 언어라고 할 수 있다.



1.2.1 접착제처럼 쓰는 파이썬

파이썬이 과학 기술 컴퓨팅계에서 성공을 하게 된 이유로 C, C++, 포트란^{FORTRAN} 코드와 통합이 쉽다는 점을 들 수 있다. 대부분의 최신 컴퓨팅 환경에서는 선형대수, 최적화, 통합, 고속 푸리에 변환 같은 알고리즘을 위해 C 라이브러리나 포트란 레거시를 공유한다. 그런 연유로 많은 회사나 국가 연구소에서 수십 년이 지난 소프트웨어를 파이썬과 함께 사용하고 있다.

프로그램은 실행 시간의 대부분을 차지하는 작은 부분의 코드와 실행 시간을 얼마 차지하지 않는 많은 양의 '글루 코드' ^{glue code} (접착제 코드, 바인딩 코드라고도 함)로 이루어져 있다. 대부분의 경우 글루 코드는 실행 시간에 영향을 주지 않을 만큼 비중이 낮다. 연산 병목을 최적화하기 위해 해당 부분을 C언어 같은 저수준 언어로 옮겨 쓴다면 유익한 결과를 얻을 수 있다.

1.2.2 한 가지 언어만 사용하자

보통 많은 기관에서 R이나 SAS 같은 좀 더 특화된 언어로 새로운 아이디어를 검증하고 프로토타입을 만들어 연구한 후 그 아이디어를 자바, C#, C++ 같은 언어를 이용하여 상용 시스템으로 포팅한다. 파이썬은 연구를 하거나 프로토타입을 만드는 데 적합한 언어인데다 실제 시스템을 개발하는 데도 적합하기에 갈수록 더 인기를 끌고 있다. 하나의 언어로 충분한데 별도의 다른 개발 환경을 유지할 필요는 없지 않은가. 연구자와 기술자가 같은 프로그래밍 도구를 사용함으로써 얻을 수 있는 장점이 많으므로 앞으로 더 많은 기관에서 파이썬을 사용하게 될 것이라 믿는다.

1.2.3 파이썬을 사용하면 안 되는 경우

파이썬이 분석 애플리케이션이나 범용 시스템을 개발하는 데 훌륭한 환경이긴 하지만 특수한 경우에는 파이썬이 아닌 다른 언어가 해답인 경우도 있다.

파이썬은 인터프리터 언어이므로 자바나 C++ 같은 컴파일 언어보다 많이 느리다. 하지만 **개발자의 시간 비용은 CPU의 시간 비용보다 비싸므로** 대개는 이런 등가 교환에 만족해한다. 그러나 실시간 거래 시스템처럼 매우 짧은 응답 시간을 필요로 하는 애플리케이션에서는 가능한 한 최고의 성능을 내고자 생산성은 떨어지지만 C++ 같은 저수준 언어로 개발을 한다.



필수 라이브러리

Essential Library

- Numpy
- Pandas
- Matplotlib
- Ipython & Jupyter
- Scipy
- Scikit-learn
- Statsmodels



1.3.1 NumPy

NumPy (넘파이)는 Numerical Python의 줄임말로, 파이썬 산술 계산의 주춧돌 같은 라이브러리다. 자료구조, 알고리즘 산술 데이터를 다루는 대부분의 과학 계산 애플리케이션에서 필요한 라이브러리를 제공한다. NumPy가 제공하는 기능은 다음과 같다.

- 빠르고 효율적인 다차원 배열 객체 ndarray
- 배열 원소를 다루거나 배열 간의 수학 계산을 수행하는 함수
- 디스크로부터 배열 기반의 데이터를 읽거나 쓸 수 있는 도구
- 선형대수 계산, 푸리에 변환, 난수 생성기
- 파이썬 확장과 C, C++ 코드에서 NumPy의 자료구조에 접근하고 계산 기능을 사용할 수 있도록 해주는 C API

고속 배열 처리 외에도 NumPy는 데이터 분석 알고리즘에 사용할 데이터 컨테이너의 역할을 한다. 수치 데이터라면 NumPy 배열은 파이썬 내장 자료구조보다 훨씬 효율적인 방법으로 데이터를 저장하고 다룰 수 있다. 또한 C나 포트란 같은 저수준 언어로 작성한 라이브러리는 NumPy 배열에 저장된 데이터를 복사하지 않고 바로 사용할 수도 있다. 따라서 파이썬을 위한 많은 산술 계산 도구는 NumPy 배열을 기본 자료구조로 가정하고 있거나 NumPy와 쉽게 연동할 수 있는 기능을 제공한다.



1.3.2 pandas

pandas(판더스)는 구조화된 데이터나 표 형식의 데이터를 빠르고 쉽고 표현적으로 다루도록 설계된 고수준의 자료구조와 함수를 제공한다. 2010년 처음 개발되어 파이썬으로 생산적이고 강력한 데이터 분석 환경을 구성하는 데 도움을 주고 있다. pandas의 주된 자료구조는 표 형태의 로우와 컬럼 이름을 가지는 DataFrame(데이터프레임)과 1차원 배열 객체인 Series(시리즈)다.

pandas는 'NumPy의 고성능, 배열 연산 아이디어'에 스프레드시트와 관계형 데이터베이스(SQL 같은)의 유연한 데이터 처리 기능을 결합한 것이다. 세련된 색인 기능을 제공하여 데이터 변형, 자르기, 취합 그리고 데이터의 부분집합을 선택할 수 있도록 해준다. 데이터를 처리하고 준비하고 다듬는 과정은 데이터 분석에서 가장 중요한 부분이므로 pandas는 이 책에서 우선적으로 집중하는 라이브러리다.

pandas 라이브러리 개발 배경을 간단히 알아보자. 나는 2008년 AQR 캐피탈 매니지먼트에서 퀀트^{quant}로 근무하는 동안 pandas 개발을 시작했는데, 그 당시 다음과 같은 요구 사항을 만족하는 도구를 찾을 수 없었기 때문이었다.

- 자동적으로 혹은 명시적으로 축의 이름에 따라 데이터를 정렬할 수 있는 자료구조. 이러한 자료구조는 잘못 정렬된 데이터에 의한 일반적인 오류를 예방하고 다양한 소스에서 가져온 다양한 방식으로 색인되어 있는 데이터를 다룰 수 있다.
- 통합된 시계열 기능
- 시계열 데이터와 비시계열 데이터를 함께 다룰 수 있는 통합 자료구조
- 메타데이터를 보존하는 산술 연산과 축약 연산
- 누락된 데이터를 유연하게 처리할 수 있는 기능
- 일반 데이터베이스(예를 들면 SQL)처럼 데이터를 합치고 관계 연산을 수행하는 기능

나는 이 모든 것을 하나로 처리할 수 있기 바랐으며 범용 소프트웨어 개발에도 사용할 수 있는 언어를 원했다. 파이썬은 이 목적에 부합하는 좋은 후보였으나 당시에는 이러한 기능을 모두 제공하는 통합 자료구조와 도구가 존재하지 않았다. pandas는 금융 문제와 사업 분석 문제를 해결할 목적으로 개발되었기에 사업 진행에 따라 생성된 데이터를 다룰 수 있는 시계열 기능과 도구가 핵심 특성이 되었다.

pandas라는 이름은 다차원으로 구조화된 데이터를 뜻하는 경제학 용어인 **패널 데이터**^{panel data}와 **파이썬 데이터 분석**^{Python data analysis}에서 따온 이름이다.



1.3.3 matplotlib

matplotlib(맷플롯립)은 그래프나 2차원 데이터 시각화를 생성하는 유명한 파이썬 라이브러리다. 존 D. 헌터가 만들었고 지금은 많은 개발 팀이 유지하고 있다. 출판물에 필요한 그래프를 만드는 데 맞춰 설계되었다. 현재 파이썬에서 사용할 수 있는 다양한 시각화 라이브러리가 존재하지만 matplotlib은 생태계 내 다른 라이브러리들과 잘 연동되어 있기에 여전히 가장 많이 사용되고 있다. 기본 시각화 도구로 가장 안전한 선택이라고 생각한다.



1.3.4 IPython과 Jupyter

IPython(아이파이썬, 인터랙티브 파이썬 Interactive Python)은 더 나은 대화형(인터랙티브) 파이썬 인터프리터를 만들 목적으로 2001년 페르난도 페레즈가 취미 프로젝트로 시작했다. 그 후 16년 동안 최신 파이썬 데이터 기술 스택에서 빠질 수 없는 가장 중요한 프로젝트로 성장했다. IPython 자체는 계산이나 데이터 분석 도구로서의 기능을 제공하지는 않지만 대화형 컴퓨팅과 소프트웨어 개발 양쪽 모두에서 생산성을 극대화할 수 있도록 설계되었다. IPython은 많은 프로그래밍 언어들의 특징인 전통적인 편집-컴파일-실행 방식 대신에 실행-탐색 방식을 장려하며 파일시스템과 운영체제 셸에도 쉽게 접근할 수 있다. 대부분의 데이터 분석 코드를 작성하는 일은 탐색적이며 반복적인 실행을 동반하므로 IPython을 이용하면 더 수월한 작업이 가능해진다.

2014년 페르난도와 IPython 팀은 언어에 상관없이 대화형 컴퓨팅 도구를 설계할 수 있는 주피터(Jupyter) 프로젝트를 발표했다. IPython 웹 노트북은 주피터 노트북으로 이름을 바꾸었고 현재 40개가 넘는 프로그래밍 언어를 지원한다. IPython 시스템은 이제 주피터에서 파이썬을 사용할 수 있게 해주는 **커널**(프로그래밍 언어 모드)로 역할을 변경했다.

IPython 자체는 이제 좀 더 큰 범위에 생산적인 대화형/탐색형 컴퓨팅 환경을 지원하는 주피터 오픈소스 프로젝트의 컴포넌트 중 하나가 되었다. 최초의 IPython은 파이썬 셸 기능을 확장하여 더 편리하게 파이썬 코드를 작성하고 테스트하고 디버깅할 수 있도록 설계되었다. 주피터 노트북에서도 IPython 시스템을 여전히 사용할 수 있는데, '노트북'이라고 하는 웹 기반의 대화형 코드 작성 환경은 다양한 프로그래밍 언어를 지원한다. IPython 셸과 주피터 노트북은 데이터를 탐색하고 시각화하는 데 특히 유용하다.

주피터 노트북 시스템은 노트북 내용을 마크다운이나 HTML로 저장할 수 있게 한다. 이를 통해 코드와 텍스트를 포함하는 문서를 생성할 수 있다. 다른 프로그래밍 언어도 주피터 환경을 위한 커널이 구현되어 있다면 파이썬 대신 주피터 환경에서 사용할 수 있다.

개인적으로 파이썬 코드를 실행하거나 디버깅, 테스트 작업을 할 때는 거의 항상 IPython을 사용한다.



1.3.5 SciPy

SciPy (사이파이)는 과학 계산 컴퓨팅 영역의 여러 기본 문제를 다루는 패키지 모음이다. 다음은 SciPy에 포함된 패키지 중 일부다.

- **scipy.integrate**
수치적분 루틴과 미분방정식 풀이법
- **scipy.linalg**
numpy.linalg에서 제공하는 것보다 더 확장된 선형대수 루틴과 매트릭스 분해
- **scipy.optimize**
함수 최적화기와 방정식의 근을 구하는 알고리즘
- **scipy.signal**
시그널 프로세싱 도구
- **scipy.sparse**
희소 행렬과 희소 선형 시스템 풀이법
- **scipy.special**
감마 함수처럼 흔히 사용되는 수학 함수를 구현한 포트란 라이브러리인 SPECFUN 래퍼
- **scipy.stats**
표준 연속/이산 확률 분포(밀도 함수, 샘플러, 연속 분포 함수)와 다양한 통계 테스트 그리고 좀 더 기술적인 통계도구

NumPy와 SciPy를 함께 사용하면 전통적인 과학 계산 애플리케이션에서 제공하는 거의 모든 기능을 대체할 수 있다.



1.3.6 scikit-learn

scikit-learn (사이킷런)은 처음 개발되기 시작한 2010년부터 파이썬 개발자를 위한 범용 머신러닝 도구로 자리 잡기 시작했다. 단 7년 만에 전 세계에서 1,500명이 넘는 사람이 프로젝트에 기여하고 있다. 다음과 같은 모델의 하위모듈을 포함한다.

- 분류: SVM, 최근접 이웃, 랜덤 포레스트, 로지스틱 회귀 등
- 회귀: 라소, 리지 회귀 등
- 클러스터링: k -평균, 스펙트럴 클러스터링 등
- 차원 축소: PCA, 특징 선택, 행렬 인수분해 등
- 모델 선택: 격자탐색, 교차검증, 행렬
- 전처리: 특징 추출, 정규화

pandas, statsmodels 그리고 IPython과 함께 scikit-learn은 파이썬이 생산적인 데이터 과학 언어로 자리매김하는 데 일등공신 역할을 했다. 이 책에 scikit-learn의 자세한 내용을 모두 담을 수는 없지만 몇몇 모델과 이 책에서 소개한 다른 도구를 어떻게 함께 이용할 수 있는지 간략하게 소개하겠다.



1.3.7 statsmodels

statsmodels은 다양한 R 언어용 회귀분석 모델을 구현한 스탠퍼드 대학의 통계학 교수인 조나단 테일러^{Jonathan Taylor}의 작업을 기반으로 만들어진 통계분석 패키지다. 스킵퍼 시볼드^{Skipper Seabold}와 조세프 퍼크톨드^{Josef Perktold}가 2010년에 새로운 statsmodels 프로젝트를 시작한 이후 수많은 사용자와 오픈소스 기여자에게 빼놓을 수 없는 프로젝트로 성장했다. 나다니엘 스미스^{Nathaniel Smith}는 R 언어의 포물러 시스템에서 착안하여 statsmodels용 포물러 또는 모델 명세 프레임워크를 제공하는 Patsy (팻시) 프로젝트를 개발했다.

scikit-learn과 비교하여 statsmodels는 전통적인 통계(주로 빈도주의적 접근)와 계량경제학 알고리즘을 포함하고 있다. 다음과 같은 하위모듈을 포함한다.

- **회귀 모델**: 선형회귀, 일반화 선형 모델, 로버스트 선형 모델, 선형 혼합효과 모델 등
- **분산분석**(ANOVA: analysis of variance)
- **시계열분석**: AR, ARMA, ARIMA, VAR 및 기타 모델
- **비모수 기법**: 커널밀도추정, 커널회귀
- **통계 모델 결과의 시각화**

statsmodels는 통계추론에 좀 더 초점을 맞추고 있다. 인자를 위한 불확실성 예측치와 p 값을 제공한다. 반면 scikit-learn은 좀 더 예측에 초점을 맞추고 있다.

scikit-learn과 함께 statsmodels을 간략히 소개하고 어떻게 이를 NumPy 및 pandas와 함께 사용하는지 알아볼 것이다.



PC별 세팅 방법

Python Setting for Windows / Mac OS / Linux

1.4.1 윈도우

먼저 아나콘다 인스톨러를 내려받는다. 이 책을 쓴 시점과 이를 읽는 시점에 따라 다소간의 차이가 있을 수 있지만 아나콘다 다운로드 페이지에 있는 안내대로 설치를 마무리한다.

설치가 완료되면 설정이 제대로 되어 있는지 확인해보자. 명령 프롬프트(cmd.exe라고도 함)를 실행하려면 ‘시작’ 메뉴를 오른 클릭하고 ‘명령’ 프롬프트를 선택하면 된다. **python**이라고 입력해서 파이썬 인터프리터를 실행하자. 방금 설치한 아나콘다 버전이 표시된 메시지를 확인할 수 있다.

```
C:\Users\wesm>python
Python 3.5.2 |Anaconda 4.1.1 (64-bit)| (default, Jul 5 2016, 11:41:13)
[MSC v.1900 64 bit (AMD64)] on win32
>>>
```

파이썬 인터프리터를 끝내려면 리눅스나 맥OS에서는 Ctrl-D를, 윈도우에서는 Ctrl-Z를 누르거나 **exit()**를 입력하고 엔터를 누르면 된다.

1.4.2 애플(OS X, macOS)

macOS용 아나콘다 인스톨러를 내려받자. Anaconda3-4.1.0-MacOSX-x86_64.pkg와 비슷한 이름의 파일이다. .pkg 파일을 더블 클릭해서 인스톨러를 실행하자. 인스톨러가 실행되면 .bash_profile 파일에 아나콘다 실행 경로가 추가된다. 이 파일은 /Users/\$USER/.bash_profile에 위치한다.

잘 설치되었는지 확인하려면 터미널 애플리케이션을 실행하고 IPython을 실행해본다.

```
$ ipython
```

파이썬 인터프리터를 끝내려면 Ctrl-D나 **exit()**를 입력하고 엔터를 누르면 된다.



1.4.3 GNU/리눅스

리눅스 환경은 사용자의 취향에 따라 설치 방법이 조금씩 다를 수 있지만 여기서는 데비안, 우분투, CentOS, 페도라 같은 배포판을 기준으로 설명하겠다. 설치 방법은 macOS와 유사하지만 아나콘다가 설치되는 방법은 조금 다르다. 인스톨러는 터미널에서 실행해야 하는 셸 스크립트로 제공된다. 32비트 또는 64비트 환경에 맞게 x86(32비트) 또는 x86_64(64비트) 인스톨러를 내려받아야 한다. 인스톨러는 Anaconda3-4.1.0-Linux-x86_64.sh 같은 이름의 파일이다. 인스톨러를 실행하려면 bash(배시)를 이용해서 다음과 같이 실행한다.

```
$ bash Anaconda3-4.1.0-Linux-x86_64.sh
```

NOTE_ 어떤 리눅스 배포판은 이미 필요한 파이썬 패키지를 가지고 있으면 apt 같은 도구를 이용해서 설치할 수 있다. 여기서는 아나콘다를 이용한 방법을 설명하고 있는데 아나콘다를 이용하면 배포판이 달라도 쉽게 재현이 가능하며 간단하게 최신 버전으로 패키지를 업그레이드할 수 있다.

라이선스에 동의하고 나면 아나콘다 파일을 어디에 복사할 것인지 물어보는데, 기본 위치인 홈 디렉터리(예를 들면 /home/\$USER/Anaconda)에 설치할 것을 추천한다.

\$PATH 환경변수에 아나콘다의 실행 파일 디렉터리인 bin/을 추가할 것인지 물어볼 수 있다. 만일 설치 후 어떤 문제가 발생한다면 직접 .bashrc(zsh을 사용한다면 .zshrc) 파일에 다음 줄을 추가할 수도 있다.

```
export PATH=/home/$USER/anaconda/bin:$PATH
```

이 과정이 끝난 후 터미널을 새로 실행하거나 source ~/.bashrc 명령으로 .bashrc 파일을 다시 실행한다.



PC별 세팅 방법

Python Setting for Windows / Mac OS / Linux

[Google Colab Setting PPT Link](#)



파이썬 패키지 설치 및 업데이트

How to Install & Update Python Packages

1.4.4 파이썬 패키지 설치하고 업데이트하기

이 책을 읽다가 아나콘다 배포판에 포함되어 있지 않은 추가 파이썬 패키지를 설치해야 할 경우 다음 명령을 이용해서 설치할 수 있다.

```
conda install package_name
```

만일 위 명령으로 설치할 수 없을 경우 pip 패키지 관리 도구를 이용해서 설치할 수 있다.

```
pip install package_name
```

conda update 명령을 이용해서 패키지를 업데이트할 수 있다.

```
conda update package_name
```

pip 역시 --upgrade 플래그를 이용해서 패키지를 업데이트할 수 있다.

```
pip install --upgrade package_name
```

위에서 소개한 명령들은 이 책을 읽는 동안 사용하게 될 기회가 있을 것이다.

CAUTION_ 패키지를 설치하기 위해 conda와 pip를 함께 사용할 수 있지만 conda 패키지를 pip 명령으로 업데이트하면 환경 설정에 문제가 발생할 수 있다. 아나콘다나 미니콘다(Miniconda)를 사용하고 있다면 conda를 이용한 업데이트를 먼저 시도하는 것이 안전하다.



1.4.6 통합 개발 환경과 텍스트 편집기

내게 기본 개발 환경에 대해서 물어오면 거의 항상 'IPython과 텍스트 편집기'라고 대답한다. 나는 보통 프로그램을 작성하고 나서 습관처럼 IPython에서 각 코드를 테스트하고 디버깅한다. 이 방법은 데이터를 쌍방향으로 다룰 수 있으며 특정한 데이터 묶음에 대한 조작이 제대로 되고 있는지 시각적으로 확인이 가능해서 상당히 유용하다. pandas와 NumPy 같은 라이브러리는 셀에서 사용하기 쉽도록 설계되었다.

하지만 어떤 사람은 소프트웨어를 개발할 때 이맥스 Emacs나 빔 Vim: Vi IMproved 같은 텍스트 편집기 대신 IDE를 선호하기도 한다. 몇 가지 IDE를 소개한다.

- **파이데브(PyDev)**: 이클립스(Eclipse) 플랫폼 기반의 IDE. 무료
- 젓브레인스의 **파이참(PyCharm)**: 상용 버전은 구독 모델이고 오픈소스 개발자들은 무료
- **PTVS**: 비주얼 스튜디오의 파이썬 도구. 윈도우 사용자용
- **스파이더(Spyder)**: 아나콘다와 함께 배포되는 IDE. 무료
- **코모도(Komodo)**: 유료

파이썬의 인기 덕분에 아톰 Atom이나 서버브라임 텍스트 2 같은 대부분의 텍스트 편집기는 파이썬을 아주 잘 지원하고 있다.

The collage features a Google search for 'vscode', various VS Code icons, and screenshots of the editor interface. Captions provide additional context for each image.

Visual Studio Code의 Codetour 플러그인으로 문서 ...
teamdable.github.io

파일:Visual Studio Code 1.35 icon.svg - ...
ko.m.wikipedia.org

VS Code, 웹버전 서비스 vscode.dev 공개 : 클리앙
clien.net

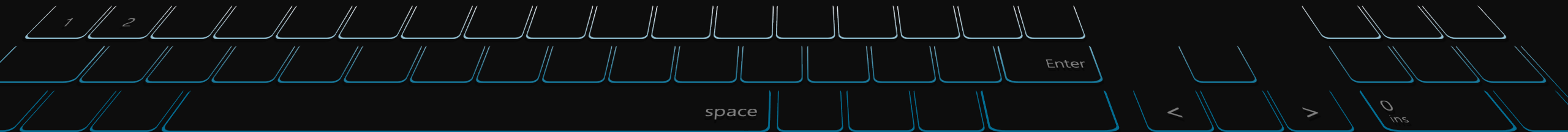
VSCode 사설망에 VSCode Server 설치
rott.k.tistory.com

특정 프로젝트에만 VS Code 설정 적용...
velog.io

11 유용한 VSCode 키보드 단축키
ichi.pro

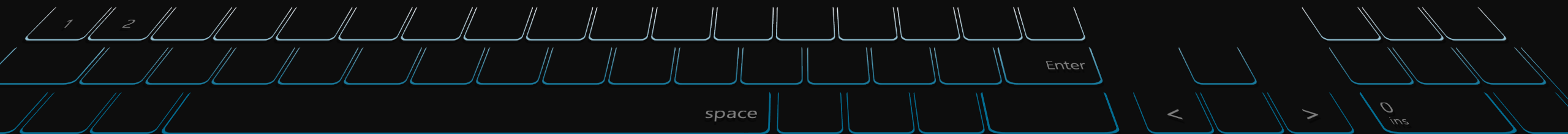
Q&A and Break Time

질의응답 및 휴식 시간 (5분)



2부 : *Python, Ipython, Jupyter Notebook*

- 1) 파이썬 인터프리터란?
- 2) Ipython 기초
- 3) Ipython 시스템
- 4) python 기초





파이썬 인터프리터란?

Python Interpreter

파이썬은 **인터프리터** 언어다. 파이썬 인터프리터는 한 번에 하나의 명령어만 실행한다. 파이썬 표준 인터프리터는 명령행에서 python을 입력해서 실행한다.

```
$ python
Python 3.6.0 | packaged by conda-forge | (default, Jan 13 2017, 23:17:12)
[GCC 4.8.2 20140120 (Red Hat 4.8.2-15)] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> a = 5
>>> print(a)
5
```

>>>는 파이썬 인터프리터의 **프롬프트**인데, 여기에 코드를 입력한다. 파이썬 인터프리터를 종료하고 명령행 프롬프트로 돌아가려면 **exit()**를 입력하거나 Ctrl-D를 누른다.

파이썬 프로그램을 실행할 때는 첫 번째 인자로 .py 파일을 넘긴다. 예를 들어 다음 내용으로 hello_world.py 파일을 작성했다고 하자.

```
print('Hello world')
```

터미널에서 다음과 같이 실행할 수 있다(hello_world.py 파일은 반드시 현재 작업하고 있는 터미널의 디렉터리에 존재해야 한다).

```
$ python hello_world.py
Hello world
```



lpython 기초

lpython Basic

2.2.1 IPython 셸 실행하기

IPython은 일반 파이썬 인터프리터를 실행시키듯이 `ipython` 명령어를 입력해서 실행시킬 수 있다.

```
$ ipython
Python 3.6.0 | packaged by conda-forge | (default, Jan 13 2017, 23:17:12)
Type "copyright", "credits" or "license" for more information.
```

```
IPython 5.1.0 -- An enhanced Interactive Python.
?          -> Introduction and overview of IPython's features.
%quickref  -> Quick reference.
help       -> Python's own help system.
object?    -> Details about 'object', use 'object??' for extra details.
```

```
In [1]: a = 5
```

```
In [2]: a
Out[2]: 5
```




IPython 기초

IPython Basic

파이썬 코드를 입력하고 Enter 키를 눌러서 실행시킬 수 있다. 파이썬에 그냥 변수 이름만 입력하면 그 객체의 문자열 표현을 출력한다.

```
In [5]: import numpy as np

In [6]: data = {i : np.random.randn() for i in range(7)}

In [7]: data
Out[7]:
{0: -0.20470765948471295,
 1: 0.47894333805754824,
 2: -0.5194387150567381,
 3: -0.55573030434749,
 4: 1.9657805725027142,
 5: 1.3934058329729904,
 6: 0.09290787674371767}
```

첫 두 줄은 파이썬 코드인데, 두 번째 줄에서 data라는 이름의 변수를 생성하고 새로 생성한 파이썬 사전형(딕셔너리)을 참조하도록 했다. 마지막 줄은 data 변수의 값을 출력한다.

대부분의 파이썬 객체는 print를 이용한 보통의 출력 결과와는 달리 좀 더 읽기 편하거나 **보기 좋은 형태로** 출력된다. 위 예제를 표준 파이썬 인터프리터에서 보면 다소 읽기 불편한 형태로 출력된다.

```
>>> from numpy.random import randn
>>> data = {i : randn() for i in range(7)}
>>> print(data)
{0: -1.5948255432744511, 1: 0.10569006472787983, 2: 1.972367135977295,
 3: 0.15455217573074576, 4: -0.24058577449429575, 5: -1.2904897053651216,
 6: 0.3308507317325902}
```

또한 IPython은 일부 코드(약간 개선된 복사/붙여넣기 기능을 통해) 아니면 전체 파이썬 스크립트를 쉽게 실행할 수 있는 기능을 제공한다. 또한 주피터 노트북을 이용해서 많은 분량의 코드를 다룰 수 있는데 나중에 더 살펴보기로 하자.



lpython 기초

lpython Basic

2.2.2 주피터 노트북 실행하기

주피터 프로젝트의 주요 구성요소 중 하나인 **노트북**은 코드, 텍스트, 데이터 시각화를 비롯한 다른 출력을 대화형으로 구성할 수 있는 대화형 문서 형식이다. 주피터 노트북은 어떤 프로그래밍 언어로도 작성 가능한 주피터 대화형 컴퓨팅 프로토콜의 구현체인 **커널**과 상호작용한다. 파이썬 주피터 커널은 IPython 시스템을 이용하여 동작한다.

주피터를 실행하려면 터미널에 `jupyter notebook` 명령을 입력하면 된다.

```
$ jupyter notebook
[I 15:20:52.739 NotebookApp] Serving notebooks from local directory:
/home/wesm/code/pydata-book
[I 15:20:52.739 NotebookApp] 0 active kernels
[I 15:20:52.739 NotebookApp] The Jupyter Notebook is running at:
http://localhost:8888/
[I 15:20:52.740 NotebookApp] Use Control-C to stop this server and shut down
all kernels (twice to skip confirmation).
Created new window in existing browser session.
```

대부분의 플랫폼에서는 `--no-browser` 옵션을 지정하지 않으면 자동으로 기본 웹 브라우저를 실행한다. 그렇지 않다면 노트북을 실행했을 때 출력되는 웹 주소(여기서는 `http://localhost:8888/`)로 접속하면 된다. 구글 크롬 브라우저에서의 실행 화면을 [그림 2-1]에서 확인할 수 있다.

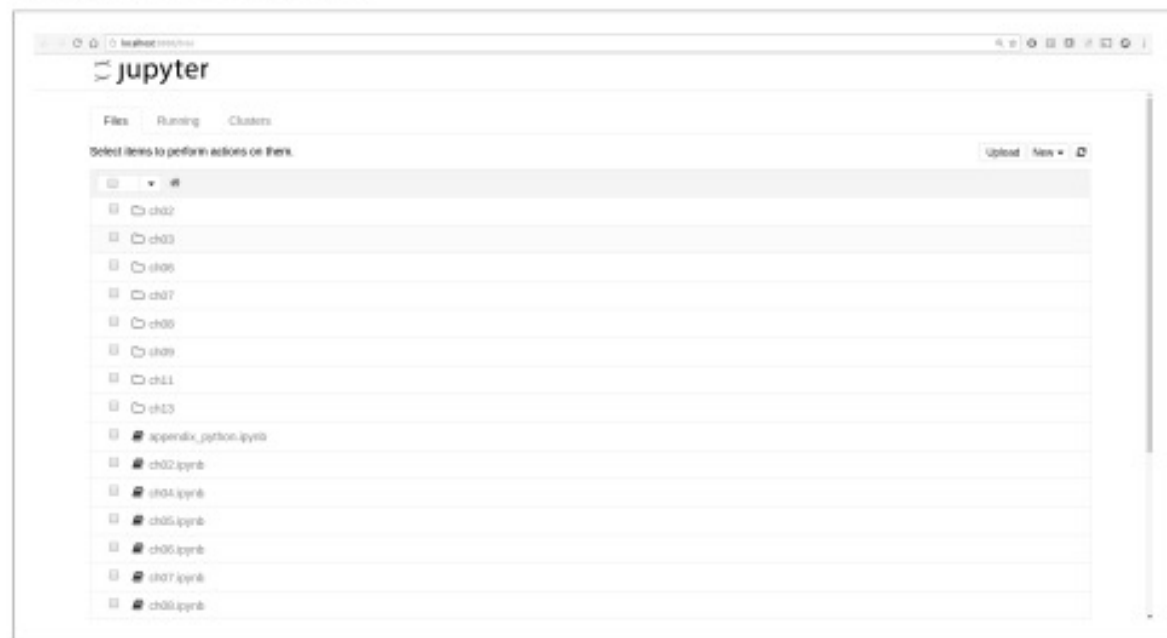
NOTE_ 많은 사람이 주피터를 로컬 개발 환경으로 사용하고 있다. 하지만 서버에 설치해두고 원격으로 접속해서 사용하는 것도 가능하다. 이 책에서는 원격 접속 설정은 다루지 않는다. 필요하다면 인터넷에서 관련 주제를 검색해보기 추천한다.



Ipython 기초

Ipython Basic

그림 2-1 주피터 노트북 랜딩 페이지



새로운 노트북을 생성하려면 New 버튼을 클릭하고 'Python 3'나 'conda[default]' 옵션을 선택한다. 그러면 [그림 2-2]와 유사한 화면을 보게 될 것이다. 처음 실행한다면 비어 있는 코드 '셀'을 클릭하고 파이썬 코드를 입력해보자. Shift-Enter를 누르면 코드를 실행할 수 있다.

그림 2-2 새 주피터 노트북 화면



노트북 파일을 저장하면(노트북의 File 메뉴에서 Save and Checkpoint 클릭) 확장자가 .ipynb인 파일이 만들어진다. 이 파일에는 현재 노트북 화면의 모든 내용이 담겨 있다. 다른 주피터 사용자가 불러와서 편집할 수 있는 파일이다. 기존 노트북 파일을 불러오려면 주피터 노트북을 실행한 경로 혹은 그 하위 폴더에 파일을 옮겨두고 랜딩 페이지에서 해당 파일을 더블 클릭하면 된다. 깃허브 저장소 [wesm/pydata-book](https://github.com/wesm/pydata-book)에 올려둔 노트북 파일로 실습해보기 바란다. [그림 2-3]을 보자.

주피터 노트북은 IPython과는 완전히 다른 경험을 주기는 하지만 이 장에서 설명하는 거의 모든 명령과 도구는 양쪽 환경 모두에서 사용할 수 있다.



Ipython 기초

Ipython Basic

2.2.3 탭 자동완성

겉으로 보기에 IPython은 표준 파이썬 인터프리터와는 조금 다르게 생겼다. 표준 파이썬 셸에 비해 가장 두드러진 개선은 **탭을 통한 자동완성 기능**으로, 대부분의 통합 개발 환경이나 대화형 데이터 분석 환경에 구현되어 있는 기능이다. 셸에서 입력을 하는 동안 탭을 누르면 네임스페이스에서 그 시점까지 입력한 내용과 맞아떨어지는 변수(객체, 함수 등)를 자동으로 찾아준다.

```
In [1]: an_apple = 27

In [2]: an_example = 42

In [3]: an<Tab>
an_apple  and          an_example  any
```

이 예제에서 IPython은 앞서 정의한 두 변수는 물론이고 파이썬 예약어인 and와 내장 함수인 any를 함께 보여주는 것을 확인할 수 있다. 물론 어떤 객체의 메서드나 속성 뒤에 마침표를 입력한 후 자동완성 기능을 활용할 수도 있다.

```
In [3]: b = [1, 2, 3]

In [4]: b.<Tab>
b.append  b.count  b.insert  b.reverse
b.clear   b.extend  b.pop     b.sort
b.copy    b.index   b.remove
```

모듈도 똑같이 동작한다.

```
In [1]: import datetime

In [2]: datetime.<Tab>
datetime.date          datetime.MAXYEAR      datetime.timedelta
datetime.datetime      datetime.MINYEAR      datetime.timezone
datetime.datetime_CAPI datetime.time          datetime.tzinfo
```

주피터 노트북과 IPython 5.0 이상 버전에서는 자동완성 목록이 일반 텍스트 출력이 아니라 드롭다운 형식으로 나타난다.

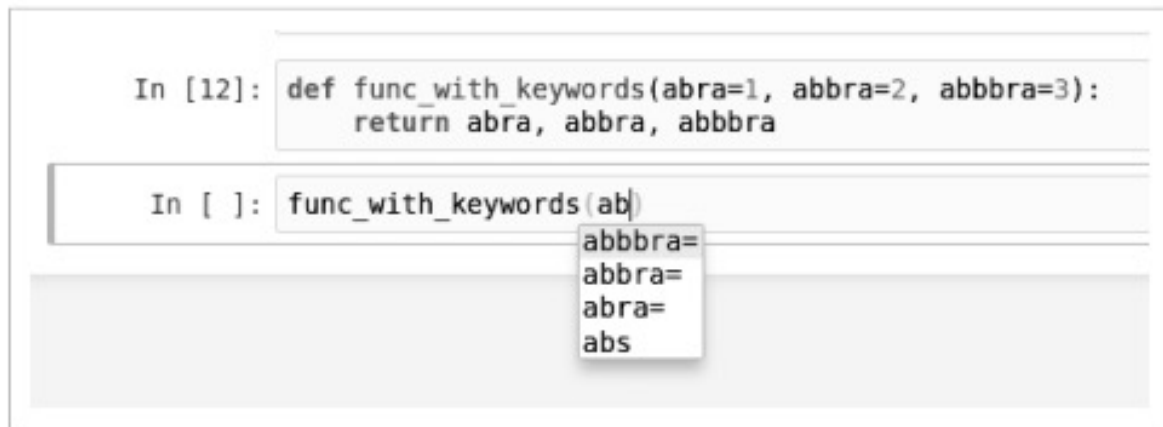


lpython 기초

lpython Basic

또한 자동완성 기능을 사용하면 함수에서 이름을 가진 인자(= 기호까지 포함해서)도 보여준다. [그림 2-4]를 참조하자.

그림 2-4 주피터 노트북에서 함수 키워드 자동완성





2.2.4 자기관찰

변수 이름 앞이나 뒤에 ? 기호를 붙이면 그 객체에 대한 일반 정보를 출력한다.

```
In [8]: b = [1, 2, 3]
```

```
In [9]: b?
```

```
Type:      list
```

```
String Form:[1, 2, 3]
```

```
Length:    3
```

```
Docstring:
```

```
list() -> new empty list
```

```
list(iterable) -> new list initialized from iterable's items
```

```
In [10]: print?
```

```
Docstring:
```

```
print(value, ..., sep=' ', end='\n', file=sys.stdout, flush=False)
```

```
Prints the values to a stream, or to sys.stdout by default.
```

```
Optional keyword arguments:
```

```
file: a file-like object (stream); defaults to the current sys.stdout.
```

```
sep: string inserted between values, default a space.
```

```
end: string appended after the last value, default a newline.
```

```
flush: whether to forcibly flush the stream.
```

```
Type:      builtin_function_or_method
```

이 기능은 객체의 **자기관찰**(인트로스펙션^{introspection})이라고 하는데, 만약 객체가 함수이거나 인스턴스 메서드라면 정의되어 있는 문서^{docstring}를 출력해준다. IPython이나 주피터에서 사용 가능한 아래와 같은 함수를 작성했다고 하자.



Ipython 기초

Ipython Basic

```
def add_numbers(a, b):
    """
    Add two numbers together

    Returns
    -----
    the_sum : type of arguments
    """
    return a + b
```

? 기호를 사용해서 문서를 출력해보자.

```
In [11]: add_numbers?
Signature: add_numbers(a, b)
Docstring:
Add two numbers together

Returns
-----
the_sum : type of arguments
File:      <ipython-input-9-6a548a216e27>
Type:      function
```

??를 사용하면 가능한 경우 함수의 소스 코드도 보여준다.

```
In [12]: add_numbers??
Signature: add_numbers(a, b)
Source:
def add_numbers(a, b):
    """
    Add two numbers together

    Returns
    -----
    the_sum : type of arguments
    """
    return a + b
File:      <ipython-input-9-6a548a216e27>
Type:      function
```

또한 ?는 표준 유닉스나 윈도우 명령행에서와 마찬가지로 IPython의 네임스페이스를 검색하는 데 사용할 수도 있다. 와일드카드(*) 기호와 함께 사용한 문자와 일치하는 모든 이름을 보여준다. 예를 들어 NumPy의 최상단 네임스페이스 안에서 load를 포함하는 모든 함수 목록을 가져올 수 있다.

```
In [13]: np.*load*?
np.__loader__
np.load
np.loads
np.loadtxt
np.pkgload
```




2.2.5 %run 명령어

%run 명령어를 사용하면 IPython 세션 안에서 파이썬 프로그램 파일을 불러와서 실행할 수 있다. 다음과 같은 ipython_script_test.py 스크립트 파일이 있다고 하자.

```
def f(x, y, z):  
    return (x + y) / z  
  
a = 5  
b = 6  
c = 7.5  
  
result = f(a, b, c)
```

이 스크립트 파일은 %run 명령을 사용해서 다음처럼 실행할 수 있다.

```
In [14]: %run ipython_script_test.py
```

스크립트 파일은 빈 네임스페이스(다른 변수가 선언되지 않았거나 아무것도 임포트되지 않은 상태)에서 실행되므로 명령행에서 python script.py 명령을 실행한 것과 동일하게 동작한다. 스크립트 파일에 정의된 모든 변수(임포트, 함수, 전역 변수)는 실행된 후에 IPython에서 접근이 가능하다.

```
In [15]: c  
Out[15]: 7.5  
  
In [16]: result  
Out[16]: 1.4666666666666666
```

만약 파이썬 스크립트에 명령행 인자(sys.argv에 저장되는)를 넘겨야 한다면 명령행에서 실행하는 것처럼 파일 경로 다음에 필요한 인자를 넘겨주면 된다.

NOTE 실행하려는 스크립트 파일에서 대화형 IPython 네임스페이스에 미리 선언된 변수에 접근해야 한다면 %run 대신 %run -i 명령을 사용한다.



lpython 기초

lpython Basic

실행 중인 코드 중지하기

%run을 통해 스크립트가 실행되고 있거나 오랜 실행 시간이 필요한 코드가 실행되고 있는 중간에 Ctrl-C를 누르면 KeyboardInterrupt 예외를 발생시킨다. 이 예외는 몇몇 특수한 경우를 제외한 거의 모든 파이썬 프로그램을 즉시 멈추도록 한다.

CAUTION_ 실행 중인 파이썬 코드가 컴파일된 확장 모듈에서 호출된 경우에는 Ctrl-C를 눌러도 프로그램이 즉각 멈추지 않는다. 이런 경우에는 프로그램의 제어권이 파이썬 인터프리터로 되돌아올 때까지 기다리거나 심각한 경우에는 운영체제의 작업 관리자 메뉴를 이용해 파이썬 프로세스를 강제로 종료해야 한다.



2.2.6 클립보드에 있는 코드 실행하기

주피터 노트북을 사용하고 있다면 아무 코드 셀에나 코드를 붙여넣기해서 실행할 수 있다. IPython 셀에서도 클립보드의 내용을 실행하는 것이 가능하다. 다른 애플리케이션에서 사용 중인 다음 코드가 있다고 하자.

```
x = 5
y = 7
if x > 5:
    x += 1

y = 8
```

이 경우 `%paste`나 `%cpaste` 매직함수를 사용할 수 있다. `%paste`는 클립보드에 있는 텍스트를 단일 블록으로 실행한다.

```
In [17]: %paste
x = 5
```

```
y = 7
if x > 5:
    x += 1

y = 8
## -- End pasted text --
```

`%cpaste`는 `%paste`와 유사하지만 코드를 붙여 넣을 때 특수한 프롬프트를 제공한다.

```
In [18]: %cpaste
Pasting code; enter '--' alone on the line to stop or use Ctrl-D.
:x = 5
:y = 7
:if x > 5:
:    x += 1
:
:    y = 8
:--
```

`%cpaste` 블록을 사용하면 실행 전에 코드를 마음껏 붙여 넣을 수 있다. 붙여 넣은 코드를 실행하기 전에 한 번 살펴보기 위해 `%cpaste` 기능을 사용할 수 있다. 실수로 잘못된 코드를 붙여 넣었다면 `Ctrl-C`를 눌러 `%cpaste` 프롬프트를 빠져나올 수 있다.



2.2.7 키보드 단축키

IPython은 이맥스 편집기나 유닉스 bash shell (베시 셸) 사용자에게 친숙한 프롬프트 이동 단축키를 제공하며 이전에 입력한 셸의 명령어 히스토리를 사용할 수 있다. [표 2-1]에 주로 사용되는 단축키를 요약해두었다. [그림 2-5]는 커서 이동 같은 몇 가지 단축키의 사용법이다.

그림 2-5 IPython 키보드 단축키

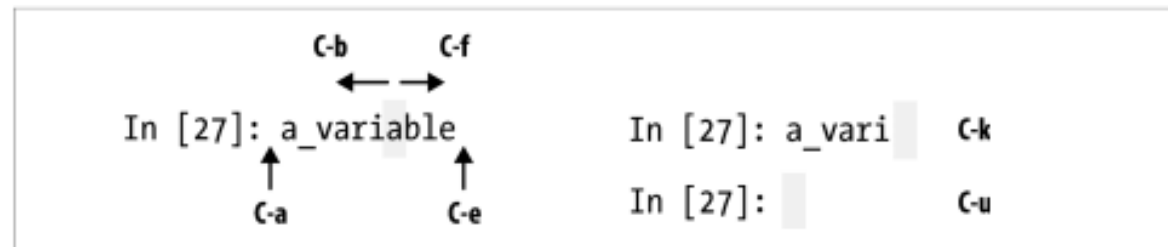


표 2-1 표준 IPython 키보드 단축키

키보드 단축키	설명
Ctrl-P 또는 위 화살표 키	명령어 히스토리를 역순으로 검색하기
Ctrl-N 또는 아래 화살표 키	명령어 히스토리에서 최근 순으로 검색하기
Ctrl-R	readline 형식의 히스토리 검색(부분 매칭)하기
Ctrl-Shift-V	클립보드에서 텍스트 붙여넣기
Ctrl-C	현재 실행 중인 코드 중단하기
Ctrl-A	커서를 줄의 처음으로 이동하기
Ctrl-E	커서를 줄의 끝으로 이동하기
Ctrl-K	커서가 놓인 곳부터 줄의 끝까지 텍스트 삭제하기
Ctrl-U	현재 입력된 모든 텍스트 지우기
Ctrl-F	커서를 앞으로 한 글자씩 이동하기
Ctrl-B	커서를 뒤로 한 글자씩 이동하기
Ctrl-L	화면 지우기

주피터 노트북은 편집과 이동에 관련된 단축키를 분리하여 제공한다. 단축키가 IPython보다 더 빠르게 진화하고 있으므로 주피터 노트북 메뉴에서 통합된 도움말을 살펴보기 권장한다.



2.2.8 매직 명령어

IPython은 파이썬 자체에는 존재하지 않는 ‘매직’ 명령어라고 하는 여러 가지 특수한 명령어를 포함하고 있다. 이 매직 명령어는 일반적인 작업이나 IPython 시스템의 동작을 쉽게 제어할 수 있도록 설계된 특수한 명령어다. 매직 명령어는 앞에 % 기호를 붙이는 형식인데, 예를 들어 IPython에서 행렬 곱셈 같은 코드가 실행된 시간을 측정하고 싶을 때는 %timeit 매직함수를 이용해서 값을 얻어올 수 있다. %timeit은 나중에 좀 더 자세히 설명하겠다.

```
In [20]: a = np.random.randn(100, 100)

In [20]: %timeit np.dot(a, a)
10000 loops, best of 3: 20.9 µs per loop
```

매직 명령어는 IPython 시스템 안에서 실행되는 명령행 프로그램으로 볼 수 있는데, 많은 매직 명령어는 추가적인 옵션을 필요로 하며 ?를 이용해서 전체 옵션을 살펴볼 수 있다.

Table 2-2. Some frequently used IPython magic commands

Command	Description
%quickref	Display the IPython Quick Reference Card
%magic	Display detailed documentation for all of the available magic commands
%debug	Enter the interactive debugger at the bottom of the last exception traceback
%hist	Print command input (and optionally output) history
%pdb	Automatically enter debugger after any exception
%paste	Execute preformatted Python code from clipboard
%cpaste	Open a special prompt for manually pasting Python code to be executed
%reset	Delete all variables/names defined in interactive namespace
%page OBJECT	Pretty-print the object and display it through a pager
%run script.py	Run a Python script inside IPython
%prun statement	Execute statement with cProfile and report the profiler output
%time statement	Report the execution time of a single statement
%timeit statement	Run a statement multiple times to compute an ensemble average execution time; useful for timing code with very short execution time
%who, %who_ls, %whos	Display variables defined in interactive namespace, with varying levels of information/verbosity
%xdel variable	Delete a variable and attempt to clear any references to the object in the IPython internals



Matplotlib Integration

One reason for IPython's popularity in analytical computing is that it integrates well with data visualization and other user interface libraries like matplotlib. Don't worry if you have never used matplotlib before; it will be discussed in more detail later in

this book. The `%matplotlib` magic function configures its integration with the IPython shell or Jupyter notebook. This is important, as otherwise plots you create will either not appear (notebook) or take control of the session until closed (shell).

In the IPython shell, running `%matplotlib` sets up the integration so you can create multiple plot windows without interfering with the console session:

```
In [26]: %matplotlib
Using matplotlib backend: Qt4Agg
```

In Jupyter, the command is a little different (Figure 2-6):

```
In [26]: %matplotlib inline
```

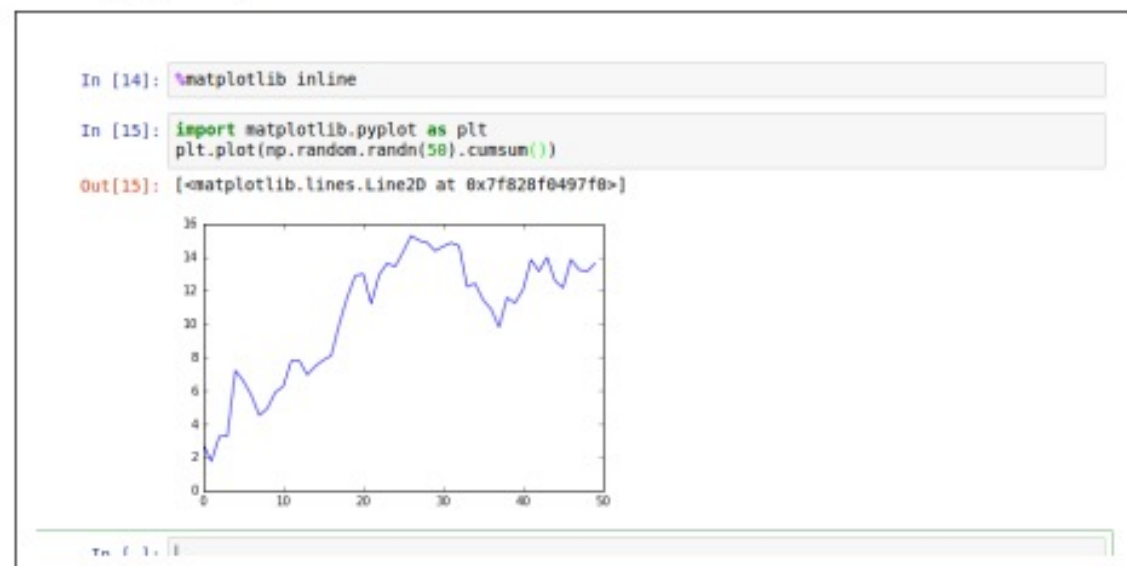


Figure 2-6. Jupyter inline matplotlib plotting



B.1 명령어 히스토리 사용하기

IPython은 이전에 실행했던 명령어를 디스크에 작은 데이터베이스 형식으로 보관하며, 다음 목적으로 사용한다.

- 검색, 자동완성, 최소한의 입력으로 이전에 실행했던 명령 재실행
- 세션 간의 명령어 히스토리 유지
- 입출력 히스토리를 파일에 기록

이 기능은 노트북보다는 셸에서 더 유용한데, 노트북은 각각의 코드 셀에서 입력과 출력을 기

B.1.1 명령어 검색과 재사용

IPython 셸에서는 이전에 실행했던 코드나 명령을 검색하고 실행할 수 있다. 이 기능은 예를 들어 `%run` 명령어나 코드를 반복 사용하는 경우가 잦을 경우 무척 유용하다. 다음과 같이 실행했다고 하자.

```
In[7]: %run first/second/third/data_script.py
```

그리고 스크립트의 실행 결과를 보니 틀린 계산이 있었다고 하자. 문제의 원인을 밝혀낸 다음 `data_script.py`를 수정하고 `%run` 명령어의 첫 일부를 입력한 다음 Ctrl-P를 누르거나 위 화살표 키를 누르면 방금 입력한 명령어와 맞아떨어지는 가장 최근에 입력한 명령어 히스토리를 검색한다. Ctrl-P나 위 화살표 키를 여러 번 누르면 계속해서 더 이전의 히스토리를 검색한다. 만일 실행하려는 명령어를 지나쳤다면 Ctrl-N이나 아래 화살표 키를 눌러 다시 돌아올 수 있다. 이 기능을 몇 번 사용해보면 어느 순간 의도하지 않고도 이 기능을 사용하는 자신을 발견하게 될 것이다!

Ctrl-R을 사용하면 bash 같은 유닉스 셸의 readline 명령어처럼 부분 순차 검색을 할 수 있다. 윈도우에서는 IPython이 readline 기능을 흉내 내도록 한다. Ctrl-R을 누르고 입력 줄에 검색하려는 몇 글자를 입력하면 이 기능을 사용할 수 있다.

```
In [1]: a_command = foo(x, y, z)

(reverse-i-search)`com': a_command = foo(x, y, z)
```

Ctrl-R을 누르면 방금 입력한 글자와 맞아떨어지는 줄을 계속해서 반복적으로 찾아준다.



B.2 운영체제와 함께 사용하기

IPython의 또 다른 중요한 기능은 운영체제 셸과 강력하게 통합되어 있다는 것이다. 즉, IPython을 종료하지 않고도 윈도우나 유닉스(리눅스, macOS) 셸에서 일반적인 명령행에서 할 수 있는 작업이 가능하다는 뜻이다. 여기에는 셸 명령어를 실행하거나, 디렉터리를 옮기거나, 명령어의 결과를 파이썬 객체(리스트나 문자열)에 저장하는 기능이 포함된다. 또한 간단한 셸 명령어 앨리어싱과 디렉터리 북마크 기능도 제공한다.

[표 B-1]에 매직함수를 요약해두었다. 기능은 잠시 후에 간단히 살펴보겠다.

표 B-1 IPython의 시스템 관련 명령어

명령	설명
!cmd	시스템 셸에서 cmd 명령어를 실행한다.
output = !cmd args	cmd 명령어를 실행하고 표준 출력(stdout) 결과를 output에 저장한다.
%alias <i>alias_name cmd</i>	시스템(셸) 명령어의 별칭을 정의한다.
%bookmark	IPython의 디렉터리 북마크 시스템 활용한다.
%cd <i>directory</i>	시스템의 작업 디렉터리를 <i>directory</i> 로 변경한다.
%pwd	현재 시스템의 작업 디렉터리를 반환한다.
%pushd <i>directory</i>	현재 디렉터리를 스택에 추가하고 새로운 디렉터리로 이동한다.
%popd	스택에 저장된 디렉터리를 꺼내서 그 디렉터리로 이동한다.
%dirs	현재 디렉터리 스택에 저장된 디렉터리 목록을 보여준다.
%dhist	접근했던 디렉터리 히스토리를 출력한다.
%env	시스템 환경 변수를 사전 타입으로 출력한다.
%matplotlib	matplotlib 통합 옵션을 설정한다.



Language Semantics

The Python language design is distinguished by its emphasis on readability, simplicity, and explicitness. Some people go so far as to liken it to “executable pseudocode.”

Indentation, not braces

Python uses whitespace (tabs or spaces) to structure code instead of using braces as in many other languages like R, C++, Java, and Perl. Consider a for loop from a sorting algorithm:

```
for x in array:
    if x < pivot:
        less.append(x)
    else:
        greater.append(x)
```

A colon denotes the start of an indented code block after which all of the code must be indented by the same amount until the end of the block.

Everything is an object

An important characteristic of the Python language is the consistency of its *object model*. Every number, string, data structure, function, class, module, and so on exists in the Python interpreter in its own “box,” which is referred to as a *Python object*. Each object has an associated *type* (e.g., *string* or *function*) and internal data. In practice this makes the language very flexible, as even functions can be treated like any other object.

Comments

Any text preceded by the hash mark (pound sign) # is ignored by the Python interpreter. This is often used to add comments to code. At times you may also want to exclude certain blocks of code without deleting them. An easy solution is to *comment out* the code:

```
results = []
for line in file_handle:
    # keep the empty lines for now
    # if len(line) == 0:
    #     continue
    results.append(line.replace('foo', 'bar'))
```

Comments can also occur after a line of executed code. While some programmers prefer comments to be placed in the line preceding a particular line of code, this can be useful at times:

```
print("Reached this line") # Simple status report
```



Python 기초

Python Basic

Function and object method calls

You call functions using parentheses and passing zero or more arguments, optionally assigning the returned value to a variable:

```
result = f(x, y, z)
g()
```

Almost every object in Python has attached functions, known as *methods*, that have access to the object's internal contents. You can call them using the following syntax:

```
obj.some_method(x, y, z)
```

Functions can take both *positional* and *keyword* arguments:

```
result = f(a, b, c, d=5, e='foo')
```

More on this later.



Variables and argument passing

When assigning a variable (or *name*) in Python, you are creating a *reference* to the object on the righthand side of the equals sign. In practical terms, consider a list of integers:

```
In [8]: a = [1, 2, 3]
```

Suppose we assign *a* to a new variable *b*:

```
In [9]: b = a
```

In some languages, this assignment would cause the data `[1, 2, 3]` to be copied. In Python, *a* and *b* actually now refer to the same object, the original list `[1, 2, 3]` (see **Figure 2-7** for a mockup). You can prove this to yourself by appending an element to *a* and then examining *b*:

```
In [10]: a.append(4)
```

```
In [11]: b  
Out[11]: [1, 2, 3, 4]
```



Figure 2-7. Two references for the same object

Understanding the semantics of references in Python and when, how, and why data is copied is especially critical when you are working with larger datasets in Python.

When you pass objects as arguments to a function, new local variables are created referencing the original objects without any copying. If you bind a new object to a variable inside a function, that change will not be reflected in the parent scope. It is therefore possible to alter the internals of a mutable argument. Suppose we had the following function:

```
def append_element(some_list, element):  
    some_list.append(element)
```

Then we have:

```
In [27]: data = [1, 2, 3]
```

```
In [28]: append_element(data, 4)
```

```
In [29]: data  
Out[29]: [1, 2, 3, 4]
```



Python 기초

Python Basic

Dynamic references, strong types

In contrast with many compiled languages, such as Java and C++, object *references* in Python have no type associated with them. There is no problem with the following:

```
In [12]: a = 5

In [13]: type(a)
Out[13]: int

In [14]: a = 'foo'

In [15]: type(a)
Out[15]: str
```

Variables are names for objects within a particular namespace; the type information is stored in the object itself. Some observers might hastily conclude that Python is not a “typed language.” This is not true; consider this example:

```
In [16]: '5' + 5
-----
TypeError                                 Traceback (most recent call last)
<ipython-input-16-f9dbf5f0b234> in <module>()
----> 1 '5' + 5
TypeError: must be str, not int
```

In some languages, such as Visual Basic, the string '5' might get implicitly converted (or *casted*) to an integer, thus yielding 10. Yet in other languages, such as JavaScript, the integer 5 might be casted to a string, yielding the concatenated string '55'. In this regard Python is considered a *strongly typed* language, which means that every object has a specific type (or *class*), and implicit conversions will occur only in certain obvious circumstances, such as the following:

```
In [17]: a = 4.5

In [18]: b = 2

# String formatting, to be visited later
In [19]: print('a is {0}, b is {1}'.format(type(a), type(b)))
a is <class 'float'>, b is <class 'int'>

In [20]: a / b
Out[20]: 2.25
```

Knowing the type of an object is important, and it's useful to be able to write functions that can handle many different kinds of input. You can check that an object is an instance of a particular type using the `isinstance` function:

```
In [21]: a = 5

In [22]: isinstance(a, int)
Out[22]: True
```

`isinstance` can accept a tuple of types if you want to check that an object's type is among those present in the tuple:

```
In [23]: a = 5; b = 4.5

In [24]: isinstance(a, (int, float))
Out[24]: True

In [25]: isinstance(b, (int, float))
Out[25]: True
```




Python 기초

Python Basic

Attributes and methods

Objects in Python typically have both attributes (other Python objects stored “inside” the object) and methods (functions associated with an object that can have access to the object’s internal data). Both of them are accessed via the syntax `obj.attribute_name`:

```
In [1]: a = 'foo'
```

```
In [2]: a.<Press Tab>
```

a.capitalize	a.format	a.isupper	a.rindex	a.strip
a.center	a.index	a.join	a.rjust	a.swapcase
a.count	a.isalnum	a.ljust	a.rpartition	a.title
a.decode	a.isalpha	a.lower	a.rsplit	a.translate
a.encode	a.isdigit	a.lstrip	a.rstrip	a.upper
a.endswith	a.islower	a.partition	a.split	a.zfill
a.expandtabs	a.isspace	a.replace	a.splitlines	
a.find	a.istitle	a.rfind	a.startswith	

Attributes and methods can also be accessed by name via the `getattr` function:

```
In [27]: getattr(a, 'split')
Out[27]: <function str.split>
```

In other languages, accessing objects by name is often referred to as “reflection.” While we will not extensively use the functions `getattr` and related functions `hasattr` and `setattr` in this book, they can be used very effectively to write generic, reusable code.

Duck typing

Often you may not care about the type of an object but rather only whether it has certain methods or behavior. This is sometimes called “duck typing,” after the saying “If it walks like a duck and quacks like a duck, then it’s a duck.” For example, you can verify that an object is iterable if it implemented the *iterator protocol*. For many objects, this means it has a `__iter__` “magic method,” though an alternative and better way to check is to try using the `iter` function:

```
def isiterable(obj):
    try:
        iter(obj)
        return True
    except TypeError: # not iterable
        return False
```

This function would return `True` for strings as well as most Python collection types:

```
In [29]: isiterable('a string')
Out[29]: True

In [30]: isiterable([1, 2, 3])
Out[30]: True

In [31]: isiterable(5)
Out[31]: False
```

A place where I use this functionality all the time is to write functions that can accept multiple kinds of input. A common case is writing a function that can accept any kind of sequence (list, tuple, ndarray) or even an iterator. You can first check if the object is a list (or a NumPy array) and, if it is not, convert it to be one:

```
if not isinstance(x, list) and isiterable(x):
    x = list(x)
```



Python 기초

Python Basic

Imports

In Python a *module* is simply a file with the *.py* extension containing Python code.

Suppose that we had the following module:

```
# some_module.py
PI = 3.14159

def f(x):
    return x + 2

def g(a, b):
    return a + b
```

If we wanted to access the variables and functions defined in *some_module.py*, from another file in the same directory we could do:

```
import some_module
result = some_module.f(5)
pi = some_module.PI
```

Or equivalently:

```
from some_module import f, g, PI
result = g(5, PI)
```

By using the *as* keyword you can give imports different variable names:

```
import some_module as sm
from some_module import PI as pi, g as gf

r1 = sm.f(pi)
r2 = gf(6, pi)
```




Table 2-3. Binary operators

Operation	Description
<code>a + b</code>	Add a and b
<code>a - b</code>	Subtract b from a
<code>a * b</code>	Multiply a by b
<code>a / b</code>	Divide a by b
<code>a // b</code>	Floor-divide a by b, dropping any fractional remainder
<code>a ** b</code>	Raise a to the b power
<code>a & b</code>	True if both a and b are True; for integers, take the bitwise AND
<code>a b</code>	True if either a or b is True; for integers, take the bitwise OR
<code>a ^ b</code>	For booleans, True if a or b is True, but not both; for integers, take the bitwise EXCLUSIVE-OR

Operation	Description
<code>a == b</code>	True if a equals b
<code>a != b</code>	True if a is not equal to b
<code>a <= b, a < b</code>	True if a is less than (less than or equal) to b
<code>a > b, a >= b</code>	True if a is greater than (greater than or equal) to b
<code>a is b</code>	True if a and b reference the same Python object
<code>a is not b</code>	True if a and b reference different Python objects

Binary operators and comparisons

Most of the binary math operations and comparisons are as you might expect:

```
In [32]: 5 - 7
Out[32]: -2
```

```
In [33]: 12 + 21.5
Out[33]: 33.5
```

```
In [34]: 5 <= 2
Out[34]: False
```

See [Table 2-3](#) for all of the available binary operators.

To check if two references refer to the same object, use the `is` keyword. `is not` is also perfectly valid if you want to check that two objects are not the same:

```
In [35]: a = [1, 2, 3]
```

```
In [36]: b = a
```

```
In [37]: c = list(a)
```

```
In [38]: a is b
Out[38]: True
```

```
In [39]: a is not c
Out[39]: True
```

Since `list` always creates a new Python list (i.e., a copy), we can be sure that `c` is distinct from `a`. Comparing with `is` is not the same as the `==` operator, because in this case we have:

```
In [40]: a == c
Out[40]: True
```

A very common use of `is` and `is not` is to check if a variable is `None`, since there is only one instance of `None`:

```
In [41]: a = None
```

```
In [42]: a is None
Out[42]: True
```



Python 기초

Python Basic

Mutable and immutable objects

Most objects in Python, such as lists, dicts, NumPy arrays, and most user-defined types (classes), are mutable. This means that the object or values that they contain can be modified:

```
In [43]: a_list = ['foo', 2, [4, 5]]
```

```
In [44]: a_list[2] = (3, 4)
```

```
In [45]: a_list
```

```
Out[45]: ['foo', 2, (3, 4)]
```

Others, like strings and tuples, are immutable:

```
In [46]: a_tuple = (3, 5, (4, 5))
```

```
In [47]: a_tuple[1] = 'four'
```

```
-----  
TypeError                                 Traceback (most recent call last)  
<ipython-input-47-b7966a9ae0f1> in <module>()  
----> 1 a_tuple[1] = 'four'  
TypeError: 'tuple' object does not support item assignment
```



Scalar Types

Python along with its standard library has a small set of built-in types for handling numerical data, strings, boolean (True or False) values, and dates and time. These “single value” types are sometimes called *scalar types* and we refer to them in this book as scalars. See [Table 2-4](#) for a list of the main scalar types. Date and time handling will be discussed separately, as these are provided by the `datetime` module in the standard library.

Table 2-4. Standard Python scalar types

Type	Description
None	The Python “null” value (only one instance of the None object exists)
str	String type; holds Unicode (UTF-8 encoded) strings
bytes	Raw ASCII bytes (or Unicode encoded as bytes)
float	Double-precision (64-bit) floating-point number (note there is no separate double type)
bool	A True or False value
int	Arbitrary precision signed integer

Numeric types

The primary Python types for numbers are `int` and `float`. An `int` can store arbitrarily large numbers:

```
In [48]: ival = 17239871
```

```
In [49]: ival ** 6
```

```
Out[49]: 26254519291092456596965462913230729701102721
```

Floating-point numbers are represented with the Python `float` type. Under the hood each one is a double-precision (64-bit) value. They can also be expressed with scientific notation:

```
In [50]: fval = 7.243
```

```
In [51]: fval2 = 6.78e-5
```

Strings

Many people use Python for its powerful and flexible built-in string processing capabilities. You can write *string literals* using either single quotes `'` or double quotes `"`:

```
a = 'one way of writing a string'
```

```
b = "another way"
```



Scalar Types

Python along with its standard library has a small set of built-in types for handling numerical data, strings, boolean (True or False) values, and dates and time. These “single value” types are sometimes called *scalar types* and we refer to them in this book as scalars. See [Table 2-4](#) for a list of the main scalar types. Date and time handling will be discussed separately, as these are provided by the `datetime` module in the standard library.

Table 2-4. Standard Python scalar types

Type	Description
None	The Python “null” value (only one instance of the None object exists)
str	String type; holds Unicode (UTF-8 encoded) strings
bytes	Raw ASCII bytes (or Unicode encoded as bytes)
float	Double-precision (64-bit) floating-point number (note there is no separate double type)
bool	A True or False value
int	Arbitrary precision signed integer

Bytes and Unicode

In modern Python (i.e., Python 3.0 and up), Unicode has become the first-class string type to enable more consistent handling of ASCII and non-ASCII text. In older versions of Python, strings were all bytes without any explicit Unicode encoding. You could convert to Unicode assuming you knew the character encoding. Let’s look at an example:

```
In [76]: val = "español"
```

```
In [77]: val
Out[77]: 'español'
```

We can convert this Unicode string to its UTF-8 bytes representation using the `encode` method:

```
In [78]: val_utf8 = val.encode('utf-8')
```

```
In [79]: val_utf8
Out[79]: b'espa\xc3\xb1ol'
```

```
In [80]: type(val_utf8)
Out[80]: bytes
```

Assuming you know the Unicode encoding of a bytes object, you can go back using the `decode` method:

```
In [81]: val_utf8.decode('utf-8')
Out[81]: 'español'
```



Scalar Types

Python along with its standard library has a small set of built-in types for handling numerical data, strings, boolean (True or False) values, and dates and time. These “single value” types are sometimes called *scalar types* and we refer to them in this book as scalars. See [Table 2-4](#) for a list of the main scalar types. Date and time handling will be discussed separately, as these are provided by the `datetime` module in the standard library.

Table 2-4. Standard Python scalar types

Type	Description
None	The Python “null” value (only one instance of the None object exists)
str	String type; holds Unicode (UTF-8 encoded) strings
bytes	Raw ASCII bytes (or Unicode encoded as bytes)
float	Double-precision (64-bit) floating-point number (note there is no separate double type)
bool	A True or False value
int	Arbitrary precision signed integer

Booleans

The two boolean values in Python are written as `True` and `False`. Comparisons and other conditional expressions evaluate to either `True` or `False`. Boolean values are combined with the `and` and `or` keywords:

```
In [89]: True and True
Out[89]: True
```

```
In [90]: False or True
Out[90]: True
```

Type casting

The `str`, `bool`, `int`, and `float` types are also functions that can be used to cast values to those types:

```
In [91]: s = '3.14159'
```

```
In [92]: fval = float(s)
```

```
In [93]: type(fval)
Out[93]: float
```

```
In [94]: int(fval)
Out[94]: 3
```

```
In [95]: bool(fval)
Out[95]: True
```

```
In [96]: bool(0)
Out[96]: False
```



Scalar Types

Python along with its standard library has a small set of built-in types for handling numerical data, strings, boolean (`True` or `False`) values, and dates and time. These “single value” types are sometimes called *scalar types* and we refer to them in this book as scalars. See [Table 2-4](#) for a list of the main scalar types. Date and time handling will be discussed separately, as these are provided by the `datetime` module in the standard library.

Table 2-4. Standard Python scalar types

Type	Description
<code>None</code>	The Python “null” value (only one instance of the <code>None</code> object exists)
<code>str</code>	String type; holds Unicode (UTF-8 encoded) strings
<code>bytes</code>	Raw ASCII bytes (or Unicode encoded as bytes)
<code>float</code>	Double-precision (64-bit) floating-point number (note there is no separate <code>double</code> type)
<code>bool</code>	A <code>True</code> or <code>False</code> value
<code>int</code>	Arbitrary precision signed integer

None

`None` is the Python null value type. If a function does not explicitly return a value, it implicitly returns `None`:

```
In [97]: a = None
```

```
In [98]: a is None
```

```
Out[98]: True
```

```
In [99]: b = 5
```

```
In [100]: b is not None
```

```
Out[100]: True
```



Scalar Types

Python along with its standard library has a small set of built-in types for handling numerical data, strings, boolean (True or False) values, and dates and time. These “single value” types are sometimes called *scalar types* and we refer to them in this book as scalars. See [Table 2-4](#) for a list of the main scalar types. Date and time handling will be discussed separately, as these are provided by the `datetime` module in the standard library.

Table 2-4. Standard Python scalar types

Type	Description
None	The Python “null” value (only one instance of the None object exists)
str	String type; holds Unicode (UTF-8 encoded) strings
bytes	Raw ASCII bytes (or Unicode encoded as bytes)
float	Double-precision (64-bit) floating-point number (note there is no separate double type)
bool	A True or False value
int	Arbitrary precision signed integer

Dates and times

The built-in Python `datetime` module provides `datetime`, `date`, and `time` types. The `datetime` type, as you may imagine, combines the information stored in `date` and `time` and is the most commonly used:

```
In [102]: from datetime import datetime, date, time
```

```
In [103]: dt = datetime(2011, 10, 29, 20, 30, 21)
```

```
In [104]: dt.day
```

```
Out[104]: 29
```

```
In [105]: dt.minute
```

```
Out[105]: 30
```

Table 2-5. Datetime format specification (ISO C89 compatible)

Type	Description
%Y	Four-digit year
%y	Two-digit year
%m	Two-digit month [01, 12]
%d	Two-digit day [01, 31]
%H	Hour (24-hour clock) [00, 23]
%I	Hour (12-hour clock) [01, 12]
%M	Two-digit minute [00, 59]
%S	Second [00, 61] (seconds 60, 61 account for leap seconds)
%w	Weekday as integer [0 (Sunday), 6]
%U	Week number of the year [00, 53]; Sunday is considered the first day of the week, and days before the first Sunday of the year are “week 0”
%W	Week number of the year [00, 53]; Monday is considered the first day of the week, and days before the first Monday of the year are “week 0”
%z	UTC time zone offset as +HHMM or -HHMM; empty if time zone naive
%F	Shortcut for %Y-%m-%d (e.g., 2012-4-18)
%D	Shortcut for %m/%d/%y (e.g., 04/18/12)



Control Flow

Python has several built-in keywords for conditional logic, loops, and other standard *control flow* concepts found in other programming languages.

if, elif, and else

The `if` statement is one of the most well-known control flow statement types. It checks a condition that, if `True`, evaluates the code in the block that follows:

```
if x < 0:
    print('It's negative')
```

An `if` statement can be optionally followed by one or more `elif` blocks and a catch-all `else` block if all of the conditions are `False`:

```
if x < 0:
    print('It's negative')
elif x == 0:
    print('Equal to zero')
elif 0 < x < 5:
    print('Positive but smaller than 5')
else:
    print('Positive and larger than or equal to 5')
```

for loops

`for` loops are for iterating over a collection (like a list or tuple) or an iterator. The standard syntax for a `for` loop is:

```
for value in collection:
    # do something with value
```

You can advance a `for` loop to the next iteration, skipping the remainder of the block, using the `continue` keyword. Consider this code, which sums up integers in a list and skips `None` values:

```
sequence = [1, 2, None, 4, None, 5]
total = 0
for value in sequence:
    if value is None:
        continue
    total += value
```

while loops

A `while` loop specifies a condition and a block of code that is to be executed until the condition evaluates to `False` or the loop is explicitly ended with `break`:

```
x = 256
total = 0
while x > 0:
    if total > 500:
        break
    total += x
    x = x // 2
```



Python 기초

Python Basic

pass

`pass` is the “no-op” statement in Python. It can be used in blocks where no action is to be taken (or as a placeholder for code not yet implemented); it is only required because Python uses whitespace to delimit blocks:

```
if x < 0:
    print('negative!')
elif x == 0:
    # TODO: put something smart here
    pass
else:
    print('positive!')
```

range

The `range` function returns an iterator that yields a sequence of evenly spaced integers:

```
In [122]: range(10)
Out[122]: range(0, 10)

In [123]: list(range(10))
Out[123]: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Both a start, end, and step (which may be negative) can be given:

```
In [124]: list(range(0, 20, 2))
Out[124]: [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]

In [125]: list(range(5, 0, -1))
Out[125]: [5, 4, 3, 2, 1]
```

As you can see, `range` produces integers up to but not including the endpoint. A common use of `range` is for iterating through sequences by index:

```
seq = [1, 2, 3, 4]
for i in range(len(seq)):
    val = seq[i]
```

Ternary expressions

A *ternary expression* in Python allows you to combine an `if-else` block that produces a value into a single line or expression. The syntax for this in Python is:

```
value = true-expr if condition else false-expr
```

Here, *true-expr* and *false-expr* can be any Python expressions. It has the identical effect as the more verbose:

```
if condition:
    value = true-expr
else:
    value = false-expr
```

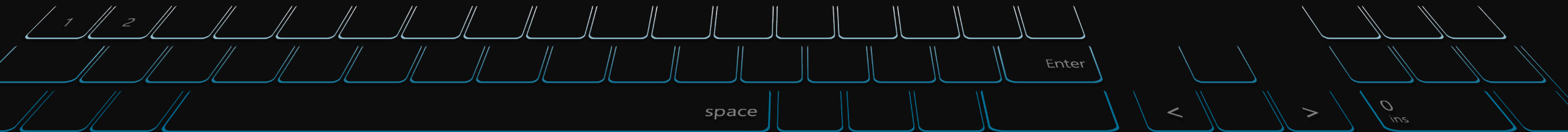
This is a more concrete example:

```
In [126]: x = 5

In [127]: 'Non-negative' if x >= 0 else 'Negative'
Out[127]: 'Non-negative'
```

Q&A

질의응답 (5분)



끝. 감사합니다.

수업 듣느라 수고하셨습니다.

